

Script - Code Walkthrough (Bash & Python)

Line-by-line explanation of the two scripts in [Script - Bandit 12 Decompression Loop](#). That note covers **why** the design is what it is; this one covers **what every construct does** and why it's written that way.

Read alongside the actual files — `unwrap.sh` and `unwrap.py` .

PART 1 — `unwrap.sh` (Bash)

The header

```
#!/usr/bin/env bash
```

The **shebang**: the first two bytes `#!` tell the kernel which interpreter to run this file with. Without it, `./unwrap.sh` fails unless invoked as `bash unwrap.sh` .

Why `/usr/bin/env bash` **and not** `/bin/bash` ? `env` searches `$PATH` for bash instead of assuming a fixed location. On macOS `/bin/bash` is an ancient version 3.2; the modern one lives elsewhere. `env` finds whichever comes first in `$PATH` — more portable.

The file also needs execute permission or the shebang never gets read:

```
chmod +x unwrap.sh    # see [[Linux - Permissions & Process Management]]
```

Safety options

```
set -uo pipefail
```

`set` changes how the shell behaves for the rest of the script. Three options exist and this uses two:

Option	Effect
<code>-e</code>	exit immediately if any command fails
<code>-u</code>	error on referencing an unset variable
<code>-o pipefail</code>	a pipeline fails if ANY stage fails, not just the last

Why `-u` matters. Without it, a typo silently becomes an empty string:

```
rm -rf "$WORKDIR/"    # correct
rm -rf "$WORDKIR/"    # typo → expands to "/" → catastrophe
```

With `-u` , the second line aborts the script instead of deleting the filesystem. This is not a hypothetical — it's the shape of several famous production incidents.

Why `pipefail` matters. By default a pipeline's exit code is that of the **last** command only:

```
false | true          # exit code 0 - "success", despite the first stage failing
```

`pipefail` makes it report failure. Relevant to the mechanics in [Linux - Piping and Redirection](#): all stages run simultaneously, so without `pipefail` an early failure is invisible.

Why `-e` is deliberately omitted. `set -e` would abort on the first non-zero exit, but this script expects commands to fail (an unrecognised format, a bad archive) and wants to handle those cases with its own error messages. `-e` would kill it before those messages ran.

Constants and arguments

```
readonly INPUT="${1:-}"
readonly MAX_LAYERS="${2:-30}"
```

`$1`, `$2` ... are the **positional parameters** — the arguments passed on the command line. `$0` is the script's own name.

`${1:-}` is **parameter expansion with a default**: "use `$1`, or an empty string if it's unset." Without it, `set -u` would abort the script when no argument is given — before the friendly usage message could run. `${2:-30}` gives `MAX_LAYERS` a default of 30.

`readonly` prevents later reassignment. On a variable that must not change, it turns a silent logic bug into an immediate error.

Naming convention: uppercase for constants and environment variables, lowercase for local working variables (`current`, `layer`, `next`). Not enforced by bash — a convention that makes scripts readable.

Input validation

```
if [[ -z "$INPUT" || ! -f "$INPUT" ]]; then
    echo "Usage: $0 <file> [max_layers]" >&2
    exit 1
fi
```

`[[ ]]` is bash's test construct (the modern replacement for `[ ]`). The operators come from [Linux - Logical Operators and Test Conditions](#):

- `-z "$INPUT"` — true if the string is **empty**
- `-f "$INPUT"` — true if the path exists **and is a regular file**
- `!` negates, `||` is OR

So: "if no argument was given, OR the argument isn't a real file, complain and stop."

`>&2` sends the message to **stderr** rather than **stdout**. This matters: it keeps error messages out of a pipeline's data stream, so `./unwrap.sh file | grep password` still works while errors go to the

terminal. Streams covered in [Linux - Piping and Redirection](#).

`exit 1` — **non-zero means failure**. Unix convention: 0 is success, anything else is an error. This is what lets `./unwrap.sh f && echo "ok"` behave correctly.

Quoting "\$INPUT" is **not optional**. Unquoted, a filename containing spaces is split into multiple arguments — the word-splitting problem documented in [Linux - Argument Parsing and Special Filenames](#).

The working directory

```
WORKDIR="$(mktemp -d "${TMPDIR:-/tmp}/unwrap.XXXXXXX")" || exit 1
```

`$( ... )` is **command substitution**: run the command, and replace the whole expression with its output. Here, `mktemp` prints the directory it created, and that path lands in `WORKDIR`.

`mktemp -d` creates a directory with a **random** name from a template. The `X` characters get replaced with random ones — at least three are required, which is why `mktemp -d david` failed in [Bandit - Level 12](#).

`${TMPDIR:-/tmp}` — same default-expansion pattern: use the `TMPDIR` environment variable if set, otherwise `/tmp`.

`|| exit 1` — *"if `mktemp` failed, exit."* Short-circuit evaluation: the right side runs only when the left side returns non-zero.

```
cp -- "$INPUT" "$WORKDIR/layer_00"
cd "$WORKDIR" || exit 1
```

`--` marks the **end of options**, so a file named `-rf` is treated as a filename rather than flags. Straight out of [Linux - Argument Parsing and Special Filenames](#).

`cd ... || exit 1` guards against the script continuing in the wrong directory if `cd` fails — which would make every subsequent relative path operate somewhere unintended.

The loop

```
while (( layer < MAX_LAYERS )); do
```

`(( ))` is **arithmetic context**. Inside it, variables need no `$`, and comparisons use `<`, `>`, `==` like other languages. Outside it, bash would compare **strings**, where `"10" < "9"` is true — the same lexicographic trap as `sort` without `-n` in [Linux - Sorting and Deduplication](#).

```
filetype="$(file -b -- "$current")"
```

`file -b` is **brief mode**: prints just the type description, without the `filename:` prefix. That prefix would otherwise have to be stripped before pattern-matching.

```
printf "[%02d] %-20s %s\n" "$layer" "$current" "${filetype:0:60}"
```

`printf` over `echo` because it gives format control:

- `%02d` — integer, zero-padded to 2 digits (`[03]` not `[3]`)
- `%-20s` — string, left-aligned, padded to 20 characters → columns line up
- `%s` — plain string
- `\n` — newline (`printf` , unlike `echo` , doesn't add one)

`${filetype:0:60}` is **substring expansion**: characters 0 through 60. `file` output can be very long; this truncates it so the trace stays one line per layer.

```
(( layer++ ))
next="layer_$(printf '%02d' "$layer")"
```

`layer++` increments inside arithmetic context. `next` becomes `layer_01` , `layer_02` ... — zero-padded so `ls` sorts them correctly.

The dispatch

```
case "$filetype" in
  "ASCII text"*)*"UTF-8"*text*)
    ...
    exit 0
    ;;
  "gzip compressed"*) gunzip -c -- "$current" > "$next" ;;
  "bzip2 compressed"*) bunzip2 -c -- "$current" > "$next" ;;
  *)
    echo "[!] Unhandled type: $filetype" >&2
    exit 1
    ;;
esac
```

`case` matches a value against **glob patterns** (not regular expressions):

- `*` matches any sequence of characters — so `"gzip compressed"` matches that phrase **anywhere** in the string
- `|` separates alternative patterns for one branch
- `)` ends a pattern, `;;` ends a branch
- `*)` at the end is the catch-all — the **default** of other languages
- `esac` is `case` backwards, closing the block

Why patterns and not exact matching? `file` returns things like `gzip compressed data`, was `"data2.bin"`, last modified: Only the distinguishing fragment matters.

Order matters. `case` takes the **first** match and stops. The catch-all must be last, or it would swallow everything.

`-c` writes to stdout so the extension check is skipped and the input is preserved; `> "$next"` redirects that stdout into the next layer's file. Redirection is resolved **before** the command runs — which is exactly what destroyed a file in [Bandit - Level 12](#) when input and output shared a name.

The tar branch

```
mapfile -t members < <(find "$extract_dir" -type f)
if (( ${#members[@]} != 1 )); then
    printf '    %s\n' "${members[@]}" >&2
    exit 1
fi
mv -- "${members[0]}" "$next"
```

Three constructs worth learning here.

`< <(command)` — **process substitution**. The inner `<(...)` runs the command and exposes its output as a temporary file; the outer `<` redirects that into `mapfile`. It looks like a typo but the two symbols do different jobs.

Why not `find ... | mapfile`? Because **each stage of a pipeline runs in a subshell**, and variables set inside a subshell vanish when it ends. `mapfile` would populate the array and then the array would disappear. Process substitution keeps `mapfile` in the current shell. This is one of the most common sources of "my variable is empty after the loop" in bash.

`mapfile -t array` reads lines into an array, one per element. `-t` strips the trailing newline from each.

Array syntax:

- `${#members[@]}` — number of elements (`#` = count, `@` = all)
- `${members[0]}` — first element (arrays are 0-indexed)
- `"${members[@]}"` — all elements, each as a separate quoted word

Post-checks

```
if [[ ! -s "$next" ]]; then
    echo "[!] Layer $layer produced an empty file - aborting." >&2
    exit 1
fi
current="$next"
done
```

`-s` is true if the file **exists and has size greater than zero**. Catching an empty result immediately is what stops the loop from spinning on nothing — the failure mode that a truncated file would otherwise cause.

`current="$next"` advances the loop: what was just produced becomes the input to the next iteration.

```
echo "[!] Hit the $MAX_LAYERS layer limit without reaching plain text." >&2
exit 1
```

Reaching this line means the `while` condition went false — the cap was hit without success. A loop driven by external data must always have a bound, or a misidentified format means it never terminates.

PART 2 — `unwrap.py` (Python)

Header and docstring

```
#!/usr/bin/env python3
"""
unwrap.py - Recursively unwrap a repeatedly compressed file.
...
"""
```

Same shebang logic. The triple-quoted string at the top of the file is the `module docstring` — not a comment: it's stored as `__doc__` and readable at runtime, which is why `argparse` can reuse it as the program's help text.

```
import argparse, bz2, gzip, io, lzma, re, sys, tarfile
```

All from the **standard library** — no `pip install` needed. That's the point of the Python version: `gzip`, `bz2`, `lzma` and `tarfile` replace the external commands the Bash version shells out to.

```
MAX_MEMBER_SIZE = 100 * 1024 * 1024 # 100 MB
```

Uppercase signals a constant by convention. Python has no `readonly` — nothing prevents reassignment; the naming is a message to humans.

Writing `100 * 1024 * 1024` instead of `104857600` is deliberate: the arithmetic **documents the unit**.

Identifying formats

```
def identify(data: bytes) -> str:
    if data[:2] == b"\x1f\x8b":
        return "gzip"
    if data[:3] == b"BZh":
        return "bzip2"
    if data[257:262] == b"ustar":
        return "tar"
```

`data: bytes` and `-> str` are **type hints**. Python doesn't enforce them at runtime — they're documentation that editors and linters can check.

`b"..."` is a **bytes literal**, not a string. Reading a file in binary mode gives bytes; comparing them to a normal string would always be false. `b"\x1f\x8b"` is two raw bytes written in hex.

`data[:2]` is **slicing**: from the start up to (not including) index 2. `data[257:262]` takes bytes 257–261 — where tar's signature lives, after the first member's header.

This function is what the `file` command does, reduced to the four formats needed. Magic numbers documented in [Linux - Nested Archives and Compression Layers](#).

```
def is_text(data: bytes) -> bool:
    if b"\x00" in data[:8192]:
        return False
    try:
        data[:8192].decode("utf-8")
        return True
    except UnicodeDecodeError:
        return False
```

Two heuristics, same ones `grep` uses to decide something is binary: **null bytes** don't appear in text, and text must **decode as UTF-8**.

`try/except` is Python's error handling: attempt the decode, and if it raises `UnicodeDecodeError`, treat it as not-text instead of crashing. Only 8 KB is checked — enough to decide, cheap on a large file.

Extracting tar safely

```
def untar(data: bytes) -> bytes:
    with tarfile.open(fileobj=io.BytesIO(data)) as tar:
        members = [m for m in tar.getmembers() if m.isfile()]
```

`io.BytesIO(data)` wraps a bytes object in a **file-like interface** — it has `.read()`, `.seek()` and so on, so any function expecting a file accepts it. Nothing touches the disk.

`with ... as tar:` is a **context manager**: it guarantees `tar` is closed when the block ends, even if an exception is raised. The equivalent of `try/finally`, in one line.

`[m for m in tar.getmembers() if m.isfile()]` is a **list comprehension** — build a list from an iterable with a filter. The loop version:

```
members = []
for m in tar.getmembers():
    if m.isfile():
        members.append(m)
```

Same result; the comprehension is the idiomatic form.

```
if member.size > MAX_MEMBER_SIZE:
    raise ValueError(f"member is {member.size} bytes - refusing (bomb guard)")
```

`f"..."` is an **f-string**: expressions inside `{}` are evaluated and inserted. `raise` throws an exception that the caller can catch.

Never `extractall()` — a member named `../../etc/cron.d/x` writes outside the target directory (CVE-2007-4559, unpatched in the standard library until Python 3.12). `extractfile()` returns a file object without writing anything to disk.

Dispatch by dictionary

```
DECOMPRESSORS = {
    "gzip": gzip.decompress,
    "bzip2": bz2.decompress,
    "xz": lzma.decompress,
    "tar": untar,
}
```

This is the Python equivalent of Bash's `case`, and it works because **functions are values** in Python. `gzip.decompress` without parentheses is the function *itself*, not a call to it. Stored in a dict, retrieved by key, then called:

```
data = DECOMPRESSORS[kind](data)
```

`DECOMPRESSORS[kind]` fetches the function; `(data)` calls it. Adding a format is one dict entry instead of a new `case` branch — the dispatch logic never changes.

The loop

```
def unwrap(data: bytes, max_layers: int = 30, verbose: bool = True) -> bytes:
    for layer in range(max_layers):
        kind = identify(data)
        if verbose:
            print(f"[{layer:02d}] {kind:<10} {len(data):>10,} bytes",
                  file=sys.stderr)
```

`max_layers: int = 30` gives the parameter a **default value** — callers can omit it. Same purpose as `${2:-30}` in Bash.

`for layer in range(max_layers)` iterates `0...max_layers-1`. **The bound is built into the loop**, so no separate check is needed — structurally safer than Bash's `while` with a manual condition.

The f-string format specifiers mirror `printf`:

- `{layer:02d}` — zero-padded to 2 digits
- `{kind:<10}` — left-aligned in 10 characters
- `{len(data):>10,}` — right-aligned in 10, with **thousands separators** (20,480)

`file=sys.stderr` sends the trace to `stderr`, so `./unwrap.py f > out.txt` captures only the payload — same separation as `>&2` in Bash.

```
try:
    data = DECOMPRESSORS[kind](data)
except Exception as exc:
    raise ValueError(f"failed to decompress {kind} at layer {layer}: {exc}")
from exc
```

Catches whatever the decompression library raises and re-raises it as a `ValueError` with useful context. `from exc` preserves the original exception in the traceback — so the message is readable *and* the underlying cause is still available for debugging.

Entry point

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__, ...)
    parser.add_argument("file", help="file to unwrap")
    parser.add_argument("--hexdump", action="store_true", ...)
    parser.add_argument("--max-layers", type=int, default=30)
    args = parser.parse_args()
```

`argparse` replaces the manual `$1 / $2` handling of Bash and gives, for free: `--help` output, type validation (`type=int` rejects non-numbers), defaults, and clear errors on bad input.

`"file"` without dashes is **positional** (required). `"--hexdump"` is an **option** (optional).

`action="store_true"` makes it a flag: present → `True`, absent → `False`.

```
if __name__ == "__main__":
    sys.exit(main())
```

`__name__` equals `"__main__"` only when the file is **run directly**. Imported as a module, it holds the module name instead — so this guard means importing `unwrap.py` from another script gives access to its functions **without executing the CLI**.

`sys.exit(main())` uses `main()`'s return value as the process exit code: 0 success, 1 failure. Same Unix convention as Bash's `exit`.

PART 3 — Equivalences

Concept	Bash	Python
Arguments	<code>\$1</code> , <code>\${2:-30}</code>	<code>argparse</code>
Constant	<code>readonly X=1</code>	<code>X = 1</code> (convention only)
Conditional	<code>[[-f "\$f"]]</code>	<code>if os.path.isfile(f):</code>
Numeric compare	<code>((a < b))</code>	<code>if a < b:</code>

Concept	Bash	Python
Dispatch	<code>case ... esac</code>	dict of functions
Array	<code>arr=(a b c), \${#arr[@]}</code>	<code>arr = [a, b, c], len(arr)</code>
Formatted output	<code>printf '%02d'</code>	<code>f"{n:02d}"</code>
Error output	<code>echo ... >&2</code>	<code>print(..., file=sys.stderr)</code>
Exit code	<code>exit 1</code>	<code>sys.exit(1)</code>
Error handling	check <code>\$?</code> after each command	<code>try/except</code>
Command output → variable	<code>x="\$(cmd)"</code>	<code>subprocess.run(..., capture_output=True)</code>

The deepest difference is error handling. Bash checks exit codes after the fact, one command at a time — easy to forget, and a forgotten check is silent. Python raises exceptions that propagate upward until something handles them: an unhandled error stops the program loudly instead of letting it continue on bad data.

That, more than syntax, is what makes Bash right for gluing tools together and Python right for anything with state, branching, or consequences.

Key Takeaway

Most of what looks like arbitrary punctuation in Bash is doing specific work: `${1:-}` prevents an abort, `>&2` keeps errors out of pipelines, `< <( ... )` avoids a subshell, `--` defuses dash-prefixed filenames, `(( ))` switches to numeric comparison. None of it is decoration, and none of it is guessable — it has to be read once, deliberately.

Python trades that density for explicitness: `argparse` instead of positional-parameter juggling, exceptions instead of exit-code checks, a dict of functions instead of glob-pattern matching. More lines, less to memorise, and errors that announce themselves.

Both scripts solve the same problem. Reading them side by side is the fastest way to see what each language is actually for.

```
Dataview (inline field '='): Error:
```

```
-- PARSING FAILED -----
```

```
> 1 | =
    | ^
```

Expected one of the following:

```
('(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated
field, number, object ('{ a: 1, b: 2 }'), string, variable
```